

Networking Fundamentals

Computer Science Fundamentals · Topic 6 — Study Note & Practice Guide

How to use this note: Read a section → close it → write the *Handnote Skeleton* box from memory → reopen and fix gaps. Run the lab commands in a terminal as you go. Each numbered section starts on a fresh page so you can work through them one at a time.

Contents

1	OSI Model	7 layers, PDUs, encapsulation, troubleshooting by layer
2	TCP vs UDP	Handshakes, reliability, flow & congestion control
3	DNS	Hierarchy, resolution flow, record types, TTL, security
4	HTTP / HTTPS	Methods, status codes, caching, CORS, HTTP/1.1 → 2 → 3
5	TLS	Certificates, chain of trust, 1.2 vs 1.3 handshake
6	Load Balancing	L4 vs L7, algorithms, health checks, HA
7	CDN	Edge caching, push vs pull, invalidation
8	Reverse Proxy	Forward vs reverse, responsibilities, Nginx
9	Browser → Server Request Lifecycle	The capstone — ties all of the above together
10	Hands-on Lab	Terminal commands to make it concrete
11	Practice Questions	15 questions with answers + 10 recall drills
12	Cheat Sheet	Ports, numbers, latency, one-line summaries

1. OSI Model

What & Why

The **OSI (Open Systems Interconnection)** model is a 7-layer *conceptual* framework describing how data moves from an application on one machine to an application on another. It is a teaching/troubleshooting model — the real internet runs on the simpler **TCP/IP model**.

Why it matters: it gives you a vocabulary. When someone says "that's a Layer 7 load balancer" or "we have a Layer 2 problem," you instantly know the scope.

The 7 Layers

Mnemonic (top→bottom): **All People Seem To Need Data Processing**

Mnemonic (bottom→top): **Please Do Not Throw Sausage Pizza Away**

#	Layer	Job	PDU (data unit)	Addressing	Examples	Devices
7	Application	Interface to user's app	Data	—	HTTP, DNS, SMTP, FTP, SSH	Firewall (L7), API gateway
6	Presentation	Translate / encrypt / compress	Data	—	TLS*, JPEG, ASCII, gzip	—
5	Session	Open, manage, close sessions	Data	—	NetBIOS, RPC, sockets	—
4	Transport	End-to-end delivery, ports, reliability	Segment (TCP) / Datagram (UDP)	Port	TCP, UDP, QUIC	L4 load balancer
3	Network	Logical addressing + routing between networks	Packet	IP address	IP, ICMP, ARP*, OSPF, BGP	Router, L3 switch
2	Data Link	Node-to-node delivery on same link, error detection	Frame	MAC address	Ethernet, Wi-Fi (802.11), PPP, VLAN	Switch, bridge, NIC
1	Physical	Raw bits over a medium	Bit	—	Cables, fiber, radio, voltages, RJ45	Hub, repeater, cable

* TLS is usually described as sitting between L4 and L7 ("Layer 5/6-ish"). ARP is technically L2/L3 boundary. Don't over-index on these edge cases.

Encapsulation — the key idea

Going **down** the stack, each layer wraps the data above it in its own header. Going **up**, each layer strips its header.

Sender	Receiver
-----	-----
L7 [Data]	[Data] L7
L4 [TCP hdr Data]	[TCP hdr Data] L4
L3 [IP hdr TCP hdr Data]	[IP hdr TCP hdr Data] L3
L2 [Eth hdr IP hdr TCP hdr Data Eth trailer]	L2
L1 1010101011101001011101010101110101...	L1
----- wire ----->	

Each wrap adds overhead: Ethernet 14 B + IP 20 B + TCP 20 B = **54 bytes minimum** before your payload.

MTU (Maximum Transmission Unit) = largest frame payload, typically **1500 bytes** on Ethernet. Bigger packets get **fragmented** (IPv4) or dropped with an ICMP "fragmentation needed" (IPv6 / Path MTU Discovery).

OSI vs TCP/IP model

OSI (7)	TCP/IP (4)	TCP/IP (5, common variant)
Application, Presentation, Session	Application	Application
Transport	Transport	Transport
Network	Internet	Network
Data Link, Physical	Network Access / Link	Data Link + Physical

Troubleshooting by layer (very practical)

Symptom	Likely layer	Check with
No link light, cable unplugged	L1	Physically check, <code>ip link</code>
Can't reach device on same LAN	L2	<code>arp -a</code> , switch/VLAN config
Can ping IP but not another subnet	L3	<code>ip route</code> , <code>traceroute</code>
Ping works, port refused	L4	<code>nc -zv host port</code> , firewall
Port open but app errors (500, cert error)	L7 / TLS	<code>curl -v</code> , logs

Handnote Skeleton — OSI

- 7 layers + mnemonic
- Table: layer → PDU → address → example → device
- Encapsulation diagram
- OSI ↔ TCP/IP mapping
- MTU = 1500

2. TCP vs UDP

Both are **Layer 4 (Transport)** protocols. Both use **ports** (16-bit, 0-65535) to identify which application on a host should receive the data.

The Core Difference

- **TCP** = a phone call. You dial, they answer, you confirm you can hear each other, then you talk. Ordered, reliable, slower to start.
- **UDP** = a postcard. You write it and drop it in the mail. No confirmation it arrived, no guarantee of order. Fast, cheap.

Comparison Table

Feature	TCP	UDP
Connection	Connection-oriented (handshake first)	Connectionless (just send)
Reliability	Guaranteed delivery (ACK + retransmit)	Best-effort, no guarantee
Ordering	In-order delivery (sequence numbers)	No ordering
Duplicate detection	Yes	No
Flow control	Yes (receiver window)	No
Congestion control	Yes (slow start, AIMD)	No (app must implement)
Header size	20-60 bytes	8 bytes
Speed / latency	Slower (handshake, ACKs, retransmits)	Faster, lower overhead
Broadcast/Multicast	No (unicast only)	Yes
Error checking	Checksum + recovery	Checksum only (detect, then drop)
Stream type	Byte stream (no message boundaries)	Message/datagram (preserves boundaries)

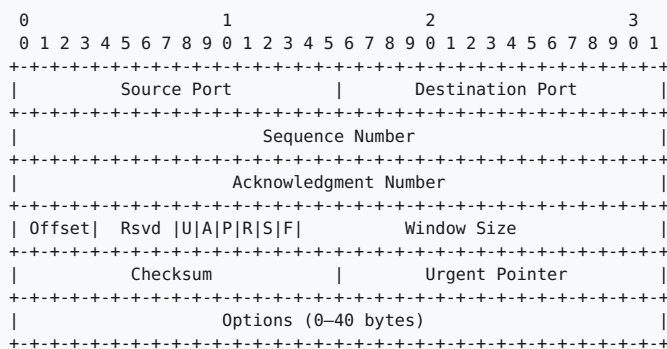
When to use which

TCP: HTTP/1.1 & HTTP/2, HTTPS, SSH, FTP, SMTP/IMAP, database connections (MySQL, Postgres), file transfer — anything where a missing byte corrupts the result.

UDP: DNS queries, DHCP, NTP, VoIP, video conferencing, live streaming, online gaming, VPNs (WireGuard), QUIC/HTTP-3, SNMP, syslog — anything where *stale data is worse than lost data*, or where you'll rebuild reliability yourself.

Key insight: in a voice call, a re-sent packet from 400 ms ago is useless — you'd rather hear a tiny glitch and stay live. That's why UDP.

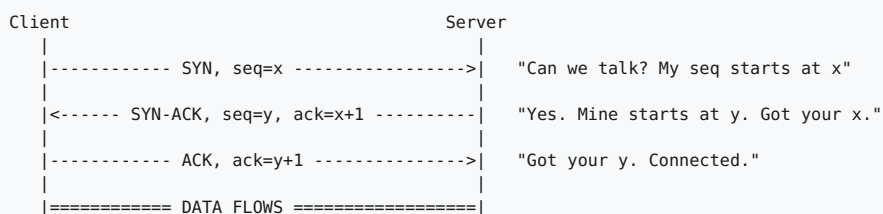
TCP Headers (what actually matters)



Flags: **URG** urgent, **ACK** acknowledgment valid, **PSH** push to app now, **RST** reset/abort connection, **SYN** synchronize (start), **FIN** finish (close).

UDP header — only 4 fields, 8 bytes total: Source Port, Dest Port, Length, Checksum. That's the whole thing.

TCP 3-Way Handshake (opening)



Cost: **1 RTT** before any data can be sent. Add TLS on top and it's 2-3 RTTs before the first byte of your actual request.

TCP 4-Way Handshake (closing)



Four steps because TCP is **full-duplex** — each direction must be closed independently. **TIME_WAIT** (typically 60 s) exists so late-arriving duplicate packets don't corrupt a new connection reusing the same ports.

Reliability mechanisms

- **Sequence numbers** — every byte is numbered, so the receiver can reorder and detect gaps.
- **ACKs** — receiver confirms "I have everything up to byte N" (cumulative ACK). **SACK** (Selective ACK) lets it say "I also have bytes 500-800" so the sender retransmits only the true gap.
- **Retransmission** — on timeout (**RTO**, based on measured RTT) or on **3 duplicate ACKs** (**fast retransmit**, doesn't wait for timeout).
- **Checksum** — corrupt segment is dropped and retransmitted.

Flow control vs Congestion control

	Flow control	Congestion control
Protects	The receiver (its buffer)	The network (routers, links)
Mechanism	Receiver advertises window size in every ACK	Sender maintains congestion window (cwnd)
Signal	Explicit ("I have 8 KB free")	Inferred (packet loss, delay, ECN)

Congestion control phases:

1. **Slow start** — cwnd starts at ~10 segments, *doubles* every RTT (exponential) until threshold or loss.
2. **Congestion avoidance** — grows by 1 segment per RTT (linear, additive increase).
3. **Fast retransmit / fast recovery** — on 3 dup ACKs, halve cwnd and continue (multiplicative decrease). This is **AIMD**.
4. **Timeout** — assume severe congestion, cwnd back to 1, restart slow start.

Common algorithms: **Reno** (classic), **CUBIC** (Linux default, loss-based), **BBR** (Google, models bandwidth+RTT instead of reacting to loss).

Gotchas worth knowing

- **Head-of-line (HOL) blocking** — one lost segment stalls *everything* behind it in the byte stream, even unrelated HTTP/2 streams. This is why HTTP/3 moved to UDP+QUIC.
- **Nagle's algorithm** — buffers small writes to avoid tiny packets. Combined with **delayed ACK** it can add ~40 ms latency. Disable with `TCP_NODELAY` for interactive apps.
- **SYN flood** — attacker sends SYNs, never ACKs, exhausting the server's half-open connection table. Defense: **SYN cookies**.
- **Ephemeral ports** — client side picks a random high port (Linux ~32768-60999). A connection is uniquely identified by the **4-tuple**: `(src IP, src port, dst IP, dst port)`.

Handnote Skeleton — TCP/UDP

- One-line analogy for each
 - Comparison table (8-10 rows)
 - 3-way handshake diagram + 4-way close
 - Flags: URG ACK PSH RST SYN FIN
 - Flow control ≠ congestion control (receiver vs network)
 - Slow start → congestion avoidance → AIMD
 - HOL blocking → motivation for QUIC
-

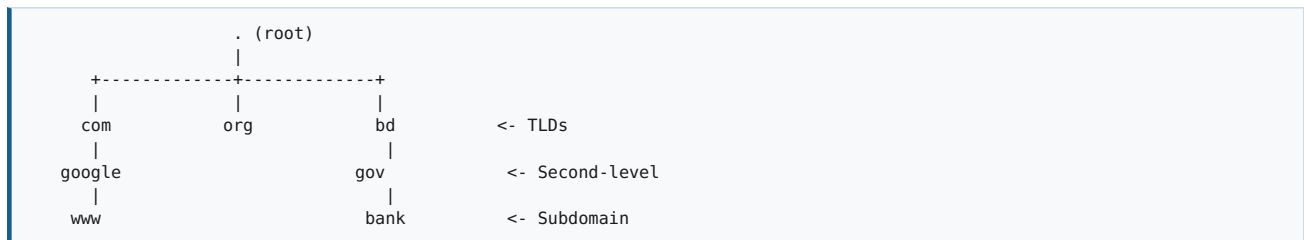
3. DNS

What & Why

DNS (Domain Name System) is the internet's phonebook: it translates human-readable names (`www.google.com`) into IP addresses (`142.250.190.78`). It's a **globally distributed, hierarchical, cached** database — no single machine holds it all.

Runs on **port 53, UDP** by default (fast, small). Falls back to **TCP** when the response exceeds 512 bytes (or 4096 with EDNS0) or for **zone transfers (AXFR)**.

The Hierarchy



A **fully qualified domain name (FQDN)** is read right-to-left: `www.google.com.` (that trailing dot is the root).

Players:

- **Root servers** — 13 logical addresses (`a.root-servers.net` ... `m`), hundreds of physical machines via **Anycast**. They know where the TLD servers are.
- **TLD servers** — `.com`, `.org`, `.net`, `.bd`, `.io`. They know which nameservers are authoritative for each domain.
- **Authoritative nameserver** — holds the actual records for a domain (e.g. Cloudflare, Route 53).
- **Recursive resolver** — the one *you* talk to (your ISP, or `8.8.8.8`, `1.1.1.1`). Does the legwork and caches.

Resolution Flow (the classic interview answer)

Looking up `www.example.com`:

1. Browser cache → hit? done.
2. OS cache / stub resolver → hit? done.
3. hosts file (/etc/hosts or C:\Windows\System32\drivers\etc\hosts)
4. Recursive resolver cache → hit? done.
↓ (cache miss – resolver now does ITERATIVE queries)
5. Resolver → Root server: "Who handles .com?" → "Ask a.gtld-servers.net"
6. Resolver → TLD server: "Who handles example.com?" → "Ask ns1.example.com" (NS + glue record)
7. Resolver → Authoritative: "What's www.example.com?" → "93.184.216.34" (A record)
8. Resolver caches it (per TTL) and returns to client.

Recursive vs iterative: the *client* makes one **recursive** query ("give me the final answer, I'll wait"). The *resolver* then makes several **iterative** queries ("just tell me who to ask next").

Glue records solve the chicken-and-egg problem when a domain's nameserver lives inside the domain itself (`ns1.example.com` serving `example.com`) — the TLD returns the IP directly.

Record Types

Type	Purpose	Example value
A	Hostname → IPv4	93.184.216.34
AAAA	Hostname → IPv6	2606:2800:220:1:248:1893::
CNAME	Alias → another <i>name</i>	www → example.com
MX	Mail servers (+ priority)	10 mail.example.com
NS	Delegates zone to nameservers	ns1.example.com
TXT	Arbitrary text — SPF, DKIM, domain verification	v=spf1 include:_spf.google.com ~all
SOA	Start of Authority — zone metadata, serial, refresh timers	one per zone
PTR	IP → hostname (reverse DNS)	used by mail servers for spam checks
SRV	Service location: host + port	_sip._tcp → 10 60 5060 sip.example.com
CAA	Which CAs may issue certs for this domain	0 issue "letsencrypt.org"
ALIAS / ANAME	CNAME-like but works at the zone apex	vendor-specific

CNAME rules to remember:

- A CNAME cannot coexist with other records for the same name.
- Therefore you **cannot** put a CNAME at the zone apex (`example.com` , which needs SOA + NS). Use ALIAS/ANAME or an A record — this is why `example.com` and `www.example.com` are configured differently.

Caching & TTL

Every record carries a **TTL** (time to live, in seconds). Caches must discard it after that.

- **Low TTL (60-300 s)** — fast failover and migrations, more DNS traffic.
- **High TTL (3600-86400 s)** — less traffic, faster lookups, but changes propagate slowly.
- **Migration tactic:** lower TTL to 60 s a day *before* the change, migrate, then raise it back.

Negative caching — `NXDOMAIN` responses are cached too (governed by the SOA minimum field).

DNS-based features

- **Round-robin DNS** — return multiple A records; clients rotate. Crude load balancing, no health awareness.
- **GeoDNS / GSLB** — return different IPs based on the resolver's location. Foundation of CDNs.
- **Anycast** — the *same* IP announced from many locations; BGP routes you to the nearest. Used by `1.1.1.1` and root servers.

Security

- **DNS is unencrypted and unauthenticated by default** → vulnerable to spoofing/cache poisoning and to ISP snooping.
- **DNSSEC** — cryptographically *signs* records so you can verify authenticity. Does **not** encrypt.
- **DoH (DNS over HTTPS)** — port 443, blends with web traffic. **DoT (DNS over TLS)** — port 853.
- **DNS hijacking / cache poisoning** — attacker injects a forged response. Mitigated by randomized source ports, 0x20 encoding, DNSSEC.
- **DNS amplification DDoS** — small spoofed query → huge response aimed at the victim.

Handnote Skeleton — DNS

- Port 53, UDP (TCP fallback >512 B / AXFR)

- Hierarchy: root → TLD → authoritative; + recursive resolver
 - 8-step resolution flow
 - Recursive (client→resolver) vs iterative (resolver→servers)
 - Record table: A, AAAA, CNAME, MX, NS, TXT, SOA, PTR, SRV, CAA
 - CNAME can't be at apex — why
 - TTL trade-off + migration trick
 - DNSSEC signs; DoH/DoT encrypt
-

4. HTTP / HTTPS

What & Why

HTTP (HyperText Transfer Protocol) is a **stateless, text-based, request/response** application-layer protocol. Client sends a request, server sends a response, connection forgets everything. Any "state" (login, cart) is bolted on via cookies, tokens, or sessions.

HTTPS = HTTP over **TLS**. Same protocol, encrypted tunnel. Default port **443** (HTTP is **80**).

Anatomy of a Request

```
POST /api/v1/users HTTP/1.1          <- method, path, version
Host: api.example.com                <- required in HTTP/1.1
User-Agent: Mozilla/5.0 ...
Content-Type: application/json
Content-Length: 42
Authorization: Bearer eyJhbGc...
Accept: application/json
                                     <- BLANK LINE separates headers from body
{"name":"Rahim","email":"r@example.com"} <- body
```

Anatomy of a Response

```
HTTP/1.1 201 Created                  <- version, status code, reason phrase
Content-Type: application/json
Content-Length: 58
Cache-Control: no-store
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Lax
Location: /api/v1/users/99
{"id":99,"name":"Rahim","email":"r@example.com"}
```

Methods

Method	Purpose	Safe?	Idempotent?	Body?
GET	Retrieve a resource	Yes	Yes	No
HEAD	Like GET, headers only	Yes	Yes	No
OPTIONS	Ask what's allowed (CORS preflight)	Yes	Yes	No
POST	Create / submit	No	No	Yes
PUT	Replace entirely	No	Yes	Yes
PATCH	Partial update	No	No (usually)	Yes
DELETE	Remove	No	Yes	Optional

- **Safe** = doesn't modify server state (read-only).
- **Idempotent** = calling it N times has the same effect as calling it once. `DELETE /users/5` twice → still deleted. `POST /orders` twice → two orders. This is why retry logic is safe on GET/PUT/DELETE but dangerous on POST.

Status Codes

Range	Meaning	Key ones
1xx	Informational	100 Continue, 101 Switching Protocols (WebSocket upgrade)
2xx	Success	200 OK, 201 Created, 202 Accepted, 204 No Content
3xx	Redirection	301 Moved Permanently, 302 Found (temp), 304 Not Modified (cache), 307/308 (preserve method)
4xx	Client error	400 Bad Request, 401 Unauthorized (not authenticated), 403 Forbidden (authenticated, not allowed), 404 Not Found, 405 Method Not Allowed, 409 Conflict, 422 Unprocessable, 429 Too Many Requests
5xx	Server error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable, 504 Gateway Timeout

Commonly confused:

- 401 vs 403 — "who are you?" vs "I know you, you still can't."
- 301 vs 302 — permanent (browser caches it, SEO transfers) vs temporary.
- 502 vs 504 — upstream gave a bad/invalid response vs upstream never responded in time.

Important Headers

Request: Host, Accept, Accept-Encoding (gzip, br), Accept-Language, Authorization, Cookie, User-Agent, Referer, If-None-Match, If-Modified-Since, Origin, Range

Response: Content-Type, Content-Length, Content-Encoding, Cache-Control, ETag, Last-Modified, Expires, Set-Cookie, Location, Access-Control-Allow-Origin, Strict-Transport-Security, Server, Retry-After

Caching

```
Cache-Control: max-age=3600          # fresh for 1 hour
Cache-Control: no-cache             # must revalidate before using (NOT "don't cache")
Cache-Control: no-store             # never write to disk/memory – use for sensitive data
Cache-Control: private              # browser only, not shared CDN/proxy caches
Cache-Control: public, max-age=31536000, immutable # fingerprinted static assets
```

Validation (conditional requests):

1. Server sends ETag: "v3-abc" (content hash) or Last-Modified: <date>.
2. On revalidation, browser sends If-None-Match: "v3-abc" or If-Modified-Since: <date>.
3. Unchanged → server replies 304 Not Modified with *no body*. Big bandwidth win.

Cache-busting pattern: name assets `app.a3f9c1.js` with `max-age=31536000, immutable`; when content changes the filename changes, so the old cache entry is simply never requested again.

Cookies & State

```
Set-Cookie: session=abc; HttpOnly; Secure; SameSite=Strict; Max-Age=3600; Path=/; Domain=example.com
```

- **HttpOnly** — JavaScript can't read it → mitigates XSS token theft.
- **Secure** — only sent over HTTPS.
- **SameSite** — `Strict` (never cross-site), `Lax` (default; sent on top-level GET navigation), `None` (requires `Secure`). Primary **CSRF** defense.

CORS (Cross-Origin Resource Sharing)

Browsers enforce the **Same-Origin Policy**: an origin = `scheme + host + port`. All three must match.

For "non-simple" requests (custom headers, `PUT / DELETE`, JSON content-type), the browser sends a **preflight**:

```
OPTIONS /api/data HTTP/1.1
Origin: https://app.example.com
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: Authorization
---
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: https://app.example.com
Access-Control-Allow-Methods: GET, PUT, POST, DELETE
Access-Control-Allow-Headers: Authorization, Content-Type
Access-Control-Allow-Credentials: true
Access-Control-Max-Age: 86400
```

CORS is a **browser** restriction, not a server one. `curl` and your backend ignore it entirely. And `Access-Control-Allow-Origin: *` cannot be combined with `Allow-Credentials: true`.

Protocol Versions — the evolution

	HTTP/1.0	HTTP/1.1	HTTP/2	HTTP/3
Year	1996	1997	2015	2022
Transport	TCP	TCP	TCP	UDP (QUIC)
Connection	New per request	Keep-alive (persistent)	Single multiplexed	Single multiplexed
Format	Text	Text	Binary frames	Binary frames
Concurrency	None	Pipelining (broken in practice)	Multiplexed streams	Multiplexed streams
Header compression	No	No	HPACK	QPACK
Encryption	Optional	Optional	Optional (mandatory in browsers)	Built into QUIC

The progression of problems solved:

1. **HTTP/1.0** — one TCP connection per request. Handshake cost on every file. Brutal.
2. **HTTP/1.1** — persistent connections + `Host` header (virtual hosting) + chunked transfer. But only ~6 parallel connections per domain, and **application-layer HOL blocking**: request 2 waits for request 1 to finish. Workarounds: domain sharding, sprite sheets, file concatenation.
3. **HTTP/2** — **multiplexing**: many independent streams over ONE TCP connection, so no application-layer HOL blocking. Binary framing, header compression (HPACK), stream prioritization, server push (now deprecated/removed). **But** TCP-level HOL blocking remains: one lost packet stalls *all* streams, because TCP delivers a single ordered byte stream.
4. **HTTP/3 / QUIC** — moves to **UDP** and rebuilds reliability *per stream* in userspace. A lost packet only stalls its own stream. Also merges the transport + TLS 1.3 handshake (**1-RTT, or 0-RTT on resumption**), and supports **connection migration** — switching Wi-Fi→mobile data keeps the connection alive via a Connection ID instead of the 4-tuple.

Handnote Skeleton — HTTP

- Stateless request/response; ports 80/443
- Request & response anatomy (start line, headers, blank line, body)
- Method table + safe vs idempotent
- Status code ranges + the confusing pairs
- Cache-Control values + ETag → 304 flow
- Cookie flags: HttpOnly, Secure, SameSite
- CORS = browser-only, preflight OPTIONS
- 1.0 → 1.1 → 2 → 3 with the problem each solved

5. TLS

What & Why

TLS (Transport Layer Security) — successor to SSL (SSL 2.0/3.0 are dead; people still say "SSL" out of habit). It provides three guarantees:

1. **Confidentiality** — encrypted, eavesdroppers see ciphertext.
2. **Integrity** — tampering is detected (MAC / AEAD tag).
3. **Authentication** — you're talking to the real server (via certificates).

Note: it does **not** hide *that* you connected, nor the destination IP, nor (usually) the hostname — **SNI** is sent in cleartext during the handshake. **ECH** (Encrypted Client Hello) is the fix, still rolling out.

Symmetric vs Asymmetric — why both

	Symmetric	Asymmetric (public key)
Keys	One shared secret	Public + private pair
Speed	Very fast	Slow (100-1000x)
Problem	How do you share the key safely?	Too slow for bulk data
Examples	AES-256-GCM, ChaCha20	RSA, ECDSA, ECDHE

TLS uses asymmetric crypto only to authenticate and agree on a shared secret, then switches to symmetric encryption for the actual data. Best of both.

Certificates & Chain of Trust

An **X.509 certificate** binds a public key to an identity, signed by a **Certificate Authority (CA)**.

Contains: subject (domain), **SAN** (Subject Alternative Names — the field browsers actually check; CN is deprecated), issuer, public key, validity dates, signature.

```
Root CA (self-signed, pre-installed in OS/browser trust store – kept offline)
├── signs → Intermediate CA
│   └── signs → Your leaf certificate (example.com)
```

The server must send the **leaf + intermediates**. The client already has the root. Verification walks up: check each signature, expiry, hostname match against SAN, and revocation.

Cert types: DV (Domain Validated — automated, Let's Encrypt), OV (Organization Validated), EV (Extended Validation). Wildcard (`*.example.com`) covers one level of subdomain only.

Revocation: CRL (big lists, slow) → OCSP (query the CA, privacy leak + latency) → **OCSP stapling** (server attaches a fresh signed proof itself — the practical answer).

TLS 1.2 Handshake (2 RTT)

```
Client                                     Server
|---- ClientHello ----->| TLS versions, cipher suites, random, SNI
|<--- ServerHello -----| chosen version + cipher, random
|<--- Certificate -----| leaf + intermediates
|<--- ServerKeyExchange (ECDHE params) -----|
|<--- ServerHelloDone -----|
|---- ClientKeyExchange (ECDHE pubkey) ----->|
|---- ChangeCipherSpec ----->|
|---- Finished (encrypted) ----->|
|<--- ChangeCipherSpec -----|
|<--- Finished (encrypted) -----|
|===== Application Data (symmetric) =====|
```

Both sides independently derive the same **master secret** from the two randoms + the key exchange, then derive session keys from it.

TLS 1.3 — what changed (2018, RFC 8446)

- **1-RTT handshake** — client guesses the key-share in ClientHello, so the whole thing finishes in one round trip. **0-RTT** on resumption (with a replay-attack caveat for non-idempotent requests).
- **Removed everything legacy**: RSA key exchange, static DH, CBC mode, RC4, MD5, SHA-1, compression, renegotiation.
- **Forward secrecy is mandatory** — only ephemeral (EC)DHE. Even if the server's private key leaks *later*, past sessions stay safe because the session key was never transmitted and is discarded.
- **AEAD only** — AES-GCM, ChaCha20-Poly1305 (encryption + integrity in one primitive).
- More of the handshake is encrypted (including the certificate).

Where TLS terminates

- **Termination at the load balancer / reverse proxy** — decrypt once at the edge, plaintext HTTP inside the trusted network. Faster, easier cert management, enables L7 routing. Add `X-Forwarded-Proto` / `X-Forwarded-For` so the app knows the original scheme and client IP.
- **Passthrough** — LB forwards encrypted bytes untouched (L4 only, can't inspect or route on paths).
- **Re-encryption / end-to-end** — decrypt at edge, re-encrypt to backend. Required for strict compliance / zero-trust.

Related hardening

- **HSTS** — `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` — browser refuses plain HTTP to this domain, blocking SSL-stripping downgrade attacks.
- **Certificate pinning** — app hardcodes the expected cert/key. Strong, but bricks your app if you rotate badly. Mostly mobile.
- **mTLS (mutual TLS)** — *client* also presents a certificate. Standard in service-to-service / zero-trust architectures.
- **Perfect Forward Secrecy** — see above; the single biggest reason to be on TLS 1.3.

Handnote Skeleton — TLS

- 3 guarantees: confidentiality, integrity, authentication
 - Asymmetric to exchange key → symmetric for bulk data (and why)
 - Cert chain: root → intermediate → leaf; SAN not CN
 - 1.2 handshake diagram (2 RTT) vs 1.3 (1 RTT / 0-RTT)
 - 1.3: forward secrecy mandatory, AEAD only, legacy removed
 - Termination vs passthrough vs re-encryption
 - HSTS, OCSP stapling, mTLS
-

6. Load Balancing

What & Why

A **load balancer (LB)** distributes incoming traffic across multiple backend servers.

Goals: scalability (add servers horizontally), high availability (route around failures), performance (avoid hot spots), maintainability (rolling deploys with zero downtime).

L4 vs L7 Load Balancing

	Layer 4 (Transport)	Layer 7 (Application)
Sees	IP + port only	Full HTTP: URL, headers, cookies, body
Decision based on	4-tuple	Path, host, method, header, cookie
Speed	Very fast, low CPU	Slower (must parse, often terminate TLS)
TLS	Passthrough	Usually terminates
Can do	NAT / forward packets	Routing rules, rewriting, caching, compression, WAF, auth
Example	AWS NLB, HAProxy (tcp mode), IPVS	AWS ALB, Nginx, Envoy, Traefik

L7 example: `/api/*` → API servers, `/static/*` → CDN origin, `/admin/*` → hardened cluster, `Host: mobile.x.com` → mobile backend. Impossible at L4.

Algorithms

Algorithm	How	Best for
Round Robin	Cycle through servers in order	Homogeneous servers, uniform requests
Weighted Round Robin	Bigger servers get more turns	Mixed hardware, canary deploys (95/5 split)
Least Connections	Send to fewest active connections	Long-lived / variable-duration requests
Weighted Least Connections	Above, adjusted by capacity	Mixed hardware + variable load
Least Response Time	Fewest connections × lowest latency	Latency-sensitive services
IP Hash	<code>hash(client IP) % N</code>	Cheap session stickiness
Consistent Hashing	Hash ring	Caches/shards — adding a node remaps only ~1/N keys, not everything
Random / Power of Two Choices	Pick 2 at random, choose the less loaded	Surprisingly near-optimal, no global state
Least Bandwidth	Fewest Mbps served	Video/file streaming

Why consistent hashing matters: with `hash % N`, going from 4→5 servers remaps ~80% of keys and destroys your cache hit rate. A hash ring remaps only the keys in one arc.

Health Checks

- **Active** — LB proactively polls `GET /health` every N seconds. Mark unhealthy after X consecutive failures, healthy again after Y successes (**hysteresis** prevents flapping).
- **Passive** — LB watches real traffic; too many 5xx/timeouts → eject the node (**outlier detection**).

- **Shallow vs deep** — shallow just returns 200 ("process is alive"); deep checks DB/cache connectivity. Deep checks are more accurate but can cascade: if the DB blips, every server marks itself unhealthy and you take down the whole fleet. A common compromise: separate `/livez` (am I running?) and `/readyz` (can I serve traffic?).

Session Persistence (Sticky Sessions)

Needed when a server holds in-memory session state.

- **Cookie-based** — LB injects a cookie naming the backend. Most reliable.
- **IP hash** — breaks with mobile IP changes and NAT'd corporate networks.

Better answer: don't need stickiness. Make servers **stateless** — put sessions in Redis or use signed JWTs. Sticky sessions cause uneven load and lose sessions when a node dies.

High Availability of the LB itself

The LB is an obvious single point of failure. Fixes:

- **Active-Passive** — standby takes over a floating **VIP** (virtual IP) via VRRP/keepalived on failure.
- **Active-Active** — multiple LBs serving simultaneously, fronted by DNS round-robin or **Anycast**.
- **DNS / GSLB** — geographic distribution across regions (this is the layer above).

Related concepts

- **Connection draining / graceful shutdown** — stop sending *new* requests to a node, let in-flight ones finish before removing it. Essential for zero-downtime deploys.
- **Rate limiting** — token bucket / leaky bucket at the LB, before traffic reaches the app.
- **Circuit breaker** — after repeated failures, stop calling a dead backend immediately (fail fast) instead of piling up timeouts.
- **Blue-green / canary** — weighted routing lets you shift 5% of traffic to a new version, watch metrics, then ramp.

Handnote Skeleton — Load Balancing

- Why: scalability, HA, performance, deploys
 - L4 vs L7 table
 - Algorithm table (esp. least-connections + consistent hashing rationale)
 - Active vs passive health checks; livez vs readyz
 - Sticky sessions — and why stateless is better
 - LB HA: active-passive VIP, active-active, anycast
 - Draining, circuit breaker, canary
-

7. CDN

What & Why

A **CDN (Content Delivery Network)** is a geographically distributed network of caching servers that serve content from a location physically near the user.

The core problem it solves is the speed of light. Dhaka → Virginia is ~13,000 km. Even at fiber speed, that's ~130 ms one way, ~260 ms RTT — *before* any processing. Add TCP handshake + TLS handshake and you've spent nearly a second before the first byte. A CDN edge node in Singapore or Dhaka cuts that to ~10–30 ms.

Benefits: lower latency, reduced origin load & bandwidth cost, DDoS absorption, higher availability, TLS termination at the edge (shorter handshake RTT), automatic image/format optimization.

Terminology

- **Origin** — your actual server, the source of truth.
- **PoP (Point of Presence)** — a datacenter location in the CDN network.
- **Edge server / node** — the caching machine inside a PoP.
- **Cache hit** — content served from edge. **Cache miss** — edge fetches from origin, stores, then serves.
- **Cache-hit ratio** — the headline metric. 95%+ is a healthy static-asset CDN.
- **Origin shield** — a designated mid-tier cache that all PoPs consult before hitting origin, so a cold cache in 200 PoPs doesn't cause 200 origin requests for the same file (**thundering herd** protection).

How a request gets routed to the nearest edge

Two mechanisms:

1. **DNS-based (GeoDNS)** — the CDN is the authoritative DNS for `cdn.example.com` and returns a different IP depending on where the resolving resolver is.
2. **Anycast** — one IP announced via BGP from every PoP; the internet's own routing delivers you to the topologically nearest one. Used by Cloudflare. Also gives free DDoS dispersion.

Push vs Pull CDN

	Pull (lazy)	Push
How	First request misses → edge fetches from origin → caches	You upload content to the CDN ahead of time
Setup	Just point DNS at CDN	Requires publishing pipeline
First user	Pays the miss penalty	Fast for everyone
Storage	Only popular content cached	You pay for everything you push
Best for	Websites, most traffic (the default)	Large infrequently-changed files: video libraries, installers, game assets

Cache control at the edge

The CDN honors origin `Cache-Control` headers, plus its own overrides.

```
Cache-Control: public, max-age=31536000, immutable # hashed static asset
Cache-Control: public, max-age=60, s-maxage=3600    # browser 1 min, CDN 1 hour
Cache-Control: private, no-store                   # never cache – user data
Vary: Accept-Encoding                             # separate cache entries per encoding
```

- `s-maxage` applies to *shared* caches (CDN/proxy) only, overriding `max-age` there.

- **Vary** tells the cache which request headers create distinct cache entries. **Vary: User-Agent** is a classic mistake — it fragments the cache into thousands of near-duplicates and destroys hit rate.

Cache key — by default the URL (+ host, + **Vary** headers). CDNs let you customize it (ignore query strings, include a cookie, etc.). Getting this wrong either poisons caches (serving user A's data to user B) or kills hit rates.

Invalidation

Cache invalidation is famously hard. Options:

1. **Versioned URLs / fingerprinting** — `app.a3f9c1.js`. **Best practice.** No invalidation needed; the URL changes so the old entry just ages out.
2. **Purge / invalidate API** — explicitly evict a path or tag. Takes seconds to minutes to propagate globally, and often rate-limited.
3. **Short TTL** — accept some staleness in exchange for simplicity.
4. **stale-while-revalidate** — serve the stale copy immediately while refreshing in the background. Great UX.

What to put on a CDN

Great fit: images, CSS, JS, fonts, video, downloads, anything static and public.

Also possible: HTML with short TTLs; API responses (GET, cacheable); **dynamic content acceleration** — even uncacheable requests benefit, because the edge terminates TLS nearby and uses a warm, optimized backbone connection to origin. **Edge compute** (Cloudflare Workers, Lambda@Edge) runs logic at the PoP.

Never cache: authenticated/personalized responses, anything with **Set-Cookie** for a session, payment flows.

Handnote Skeleton — CDN

- Problem = speed of light + origin load
 - Origin / PoP / edge / hit / miss / hit-ratio / origin shield
 - Routing: GeoDNS vs Anycast
 - Push vs pull table
 - s-maxage, Vary, cache key
 - Invalidation: fingerprinted URLs > purge API
 - Edge compute, dynamic acceleration
-

8. Reverse Proxy

What & Why

A **reverse proxy** sits *in front of servers* and forwards client requests to them. Clients think they're talking to the proxy; they never see the backend.



Forward vs Reverse Proxy — the classic confusion

	Forward Proxy	Reverse Proxy
Sits in front of	Clients	Servers
Hides	The client from the server	The servers from the client
Configured by	The client/user	The server operator
Client knows about it?	Yes (it's configured in the browser/OS)	No (it looks like the origin)
Purpose	Anonymity, bypass geo-blocks, corporate filtering, client-side caching	Load balancing, TLS termination, caching, security
Examples	Corporate proxy, VPN-ish, Squid, Tor	Nginx, HAProxy, Envoy, Cloudflare, Traefik

One-line memory hook: a forward proxy protects the *client's* identity; a reverse proxy protects the *server's*.

What a reverse proxy actually does

1. **Load balancing** — distribute across backends (most reverse proxies are also L7 LBs).
2. **TLS termination** — handle certs and decryption once, at one place.
3. **Caching** — serve repeated responses without touching the app.
4. **Compression** — gzip/brotli, offloaded from the app.
5. **Security** — hide backend topology & IPs, WAF rules, DDoS/rate limiting, IP allowlists, block bad bots.
6. **Static file serving** — Nginx serves files far faster than a Python/Node process.
7. **Request routing & rewriting** — path-based routing to microservices, URL rewrites, header injection.
8. **Authentication offload** — validate JWT/OAuth at the edge before hitting app code.
9. **Observability** — one place for access logs, metrics, tracing headers.
10. **Protocol translation** — accept HTTP/3 from clients, speak HTTP/1.1 to a legacy backend.

Reverse Proxy vs Load Balancer vs API Gateway

They overlap heavily; the distinction is *emphasis*, not hard boundaries.

- **Load balancer** — primary job is *distributing* traffic across identical replicas. Can exist with just one purpose.
- **Reverse proxy** — primary job is *intermediation*: it's useful even with a **single** backend (TLS, caching, static files, security).
- **API gateway** — a reverse proxy specialized for APIs: auth, rate limits per key, quotas, request/response transformation, aggregation, versioning, developer portal.

Nginx is all three depending on how you configure it.

Minimal Nginx config (worth memorizing the shape)

```
upstream backend {
    least_conn;
    server 10.0.1.10:8080 weight=3 max_fails=3 fail_timeout=30s;
    server 10.0.1.11:8080;
    server 10.0.1.12:8080 backup;
    keepalive 32;
}

server {
    listen 443 ssl http2;
    server_name example.com;

    ssl_certificate     /etc/ssl/fullchain.pem;
    ssl_certificate_key /etc/ssl/privkey.pem;
    ssl_protocols       TLSv1.2 TLSv1.3;

    gzip on;
    gzip_types text/css application/javascript application/json;

    location /static/ {
        root /var/www;
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    location / {
        proxy_pass http://backend;
        proxy_set_header Host          $host;
        proxy_set_header X-Real-IP      $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }
}

server {                                # redirect plain HTTP
    listen 80;
    server_name example.com;
    return 301 https://$host$request_uri;
}
```

Why the `X-Forwarded-*` headers matter: once the proxy terminates TLS, the backend sees the *proxy's* IP and thinks the connection is plain HTTP. These headers carry the truth. Your app must be configured to trust them (and *only* from the proxy — otherwise clients can spoof their IP).

Handnote Skeleton — Reverse Proxy

- Diagram: client → proxy → backends
- Forward vs reverse table + the one-line hook
- 10 responsibilities (at least list 6)
- RP vs LB vs API gateway distinction
- Nginx config shape: upstream block + server block + proxy_set_header
- X-Forwarded-For / -Proto and why

9. Browser → Server Request Lifecycle

This is the capstone. It's the single most common systems-design/interview question ("What happens when you type google.com and hit Enter?") because it touches **every** topic above. Learn this and the rest locks into place.

Stage 1 — Input processing

1. Browser checks whether the input is a **URL or a search query**.
2. **URL parsing**: `https://www.example.com:443/products?id=5#reviews`
- scheme `https` · host `www.example.com` · port `443` · path `/products` · query `?id=5` · fragment `#reviews`
(**never sent to the server**)
3. **Punycode / IDN conversion** for non-ASCII domains (`বাংলা.com` → `xn--...`).
4. **HSTS check** — if the domain is in the preload list or previously sent HSTS, the browser rewrites `http://` → `https://` *before any network activity*.

Stage 2 — DNS resolution

5. Check caches in order: **browser cache** → **OS cache** → `/etc/hosts` → **recursive resolver cache**.
6. On miss, resolver does iterative lookups: **root** → **TLD** → **authoritative** (see §3).
7. Returns an IP. Browser may also start **speculative DNS prefetch** for other domains on the page.

Stage 3 — Establishing the connection

8. **Routing decision**: is the destination on my local subnet? Compare against the subnet mask. If not, send to the **default gateway**.
9. **ARP** — the machine needs the gateway's **MAC address** to build the Ethernet frame. Broadcast "who has 192.168.1.1?" → gateway replies with its MAC. (Cached in the ARP table.)
10. Packet travels: your NIC → Wi-Fi/router → **NAT** rewrites your private IP to the public one → ISP → BGP-routed across autonomous systems → destination network → possibly a CDN edge or load balancer rather than the true origin.
11. **TCP 3-way handshake** — SYN / SYN-ACK / ACK. (**1 RTT**)
12. **TLS handshake** — ClientHello (with SNI + ALPN advertising `h2/h3`) → certificate → key agreement → Finished. (**1 RTT on TLS 1.3, 2 on 1.2**). Browser validates chain, expiry, hostname vs SAN, revocation.
 - If HTTP/3: steps 11 and 12 collapse into a single QUIC handshake over UDP.

Stage 4 — The request

13. Browser sends the HTTP request: method, path, `Host`, cookies for that domain, `Accept-*`, `Authorization`, conditional headers (`If-None-Match`).
14. Before this, it may short-circuit entirely from the **HTTP cache** — a fresh entry means no network request at all; a stale one means a conditional request that may return `304`.

Stage 5 — Server side

15. Hits the **CDN edge** → cache hit? serve immediately. Miss → forward to origin (possibly via origin shield).
16. **Load balancer** picks a healthy backend (L4 or L7 rule).
17. **Reverse proxy** may terminate TLS, serve static files, or route by path.
18. **App server** runs: routing → middleware (auth, rate limit) → controller → **cache (Redis)** → **database** → serialize response.
19. Response travels back with status code, headers, body — likely gzip/brotli compressed.

Stage 6 — Browser rendering

20. **Parse HTML** → build the **DOM** tree (incrementally, as bytes arrive).
21. **Parse CSS** → build the **CSSOM**. CSS is **render-blocking**.
22. **JavaScript** — a plain `<script>` blocks parsing. `async` = fetch in parallel, execute whenever ready. `defer` = fetch in parallel, execute in order after parsing completes.
23. **Render tree** = DOM + CSSOM, excluding `display:none` nodes.
24. **Layout / reflow** — compute exact geometry (position + size) of every box.
25. **Paint** — fill in pixels: text, colors, borders, shadows.
26. **Compositing** — combine layers (GPU-accelerated); `transform` and `opacity` animate here, which is why they're cheap.
27. Subresources (images, fonts, more scripts) trigger **their own full lifecycle**, from step 5 onward — often to different domains.
28. `DOMContentLoaded` fires (HTML parsed + deferred scripts done), then `load` (all subresources done).

The condensed version (say this out loud until fluent)

URL parse → HSTS → DNS (browser/OS/resolver → root → TLD → authoritative) → ARP + routing to gateway → TCP handshake → TLS handshake → HTTP request → CDN → load balancer → reverse proxy → app server → cache/DB → response → HTML parse → DOM + CSSOM → render tree → layout → paint → composite → subresources repeat.

Where the time actually goes

Phase	Typical cost	How to cut it
DNS	20-120 ms (0 if cached)	Longer TTLs, <code>dns-prefetch</code> , fast resolver
TCP handshake	1 RTT	Keep-alive, connection reuse, CDN (shorter RTT)
TLS handshake	1-2 RTT	TLS 1.3, session resumption, OCSP stapling, CDN
Server processing (TTFB)	5 ms - 2 s	Caching, DB indexes, async work
Content download	size ÷ bandwidth	Compression, HTTP/2, smaller assets
Rendering	50-500 ms	Reduce blocking CSS/JS, defer, critical CSS

Preconnect hints let the browser do steps 5-12 early:

```
<link rel="dns-prefetch" href="https://cdn.example.com">
<link rel="preconnect" href="https://cdn.example.com" crossorigin>
<link rel="preload" href="/fonts/main.woff2" as="font" crossorigin>
```

Handnote Skeleton — Lifecycle

- 6 stages: input → DNS → connection → request → server → render
- Memorize the condensed one-liner
- Note where each earlier topic (§1-§8) appears
- Render pipeline: DOM + CSSOM → render tree → layout → paint → composite
- `async` vs `defer`
- Latency budget table

10. Hands-on Lab

Run these in a terminal (Linux/macOS; on Windows use WSL or the noted equivalents). This is where the concepts stop being abstract.

DNS

```
dig example.com          # full DNS answer with TTL
dig example.com +short    # just the IP
dig example.com A +trace  # WATCH THE FULL root → TLD → authoritative walk. Do this one.
dig example.com MX        # mail servers
dig example.com NS        # nameservers
dig example.com TXT        # SPF/DKIM records
dig @1.1.1.1 example.com  # query a specific resolver
dig -x 8.8.8.8            # reverse lookup (PTR)
nslookup example.com      # simpler, works on Windows too
```

Exercise: run `dig google.com +trace` and identify the root server, the TLD server, and the authoritative server in the output.

Connectivity & Routing

```
ping example.com          # ICMP reachability + RTT
traceroute example.com    # every hop along the path (Windows: tracert)
mtr example.com           # traceroute + ping combined, live
ip route                  # your routing table (or: route -n)
ip addr                   # your interfaces and IPs (or: ifconfig)
arp -a                   # MAC→IP table – this is Layer 2 in front of you
```

Exercise: run `traceroute` to a server on another continent and count the hops. Note where latency jumps — that's usually the undersea cable.

Ports & Sockets

```
ss -tuln                 # listening TCP/UDP sockets (modern netstat)
ss -tn state established # live TCP connections
nc -zv example.com 443    # is port 443 open?
lsof -i :8080             # which process owns port 8080
```

HTTP

```
curl -v https://example.com # see request + response headers AND TLS details
curl -I https://example.com  # HEAD – headers only
curl -L https://example.com  # follow redirects (watch the 301 chain)
curl --http2 -I https://example.com # confirm HTTP/2
curl --http3 -I https://cloudflare.com # confirm HTTP/3 (needs a curl built with HTTP/3)
curl -H "Accept-Encoding: gzip" -I https://example.com
curl -X POST -d '{"a":1}' -H "Content-Type: application/json" https://httpbin.org/post
curl -w "dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer} total:%{time_total}\n" -o /dev/null -s https://example.com
```

That last one is the most valuable command in this note — it shows you the exact latency breakdown of every stage in §9.

TLS

```
openssl s_client -connect example.com:443 -servername example.com
openssl s_client -connect example.com:443 -tls1_3 # force a version
echo | openssl s_client -connect example.com:443 2>/dev/null | openssl x509 -noout -dates -subject -issuer
curl -vI https://expired.badssl.com # see a real cert failure
```

Try the badssl.com family in a browser: `expired.`, `self-signed.`, `wrong.host.`, `untrusted-root.` — each shows a different validation step failing.

Packet-level


```
sudo tcpdump -i any -n port 53          # watch DNS queries fly
sudo tcpdump -i any -n 'tcp[tcpflags] & (tcp-syn|tcp-fin) != 0'  # see handshakes
```

Or use **Wireshark** with filter `tcp.flags.syn==1 && tcp.flags.ack==0` and literally watch the 3-way handshake.

Browser DevTools

Open **Network tab** → reload a page → click any request → **Timing** panel. You will see, labeled: Queueing, DNS Lookup, Initial connection, SSL, Waiting (TTFB), Content Download. That's \$9 rendered visually. Also check the **Protocol** column (h2/h3) and the **Size** column (`disk cache` = a cache hit).

11. Practice Questions

Cover the answers. Write yours first, then compare.

Conceptual

Q1. Why does TCP need a 3-way handshake instead of 2?

Answer: Both sides must agree on initial sequence numbers *and* confirm the other side's number was received. Two messages would only confirm one direction. The third message tells the server its own sequence number arrived — otherwise the server can't know the client is really there (and half-open connections would be trivially exploitable).

Q2. Your website loads fine from your laptop but users report it's down. Name three checks, one per layer.

Answer: L3: `traceroute` / `ping` — is the route reachable from their region? L4: `nc -zv host 443` — is the port open from outside, or is a firewall/security group blocking? L7: `curl -v` — HTTP status, cert validity, `Host` header routing. Also check DNS: your laptop may have a stale cached record pointing at the old IP.

Q3. Why can't you put a CNAME at the zone apex?

Answer: The apex must carry SOA and NS records, and DNS forbids a CNAME from coexisting with any other record for the same name. Workaround: ALIAS/ANAME records (vendor-specific, resolved server-side) or a plain A record.

Q4. Explain the difference between `401` and `403`, and between `502` and `504`.

Answer: `401` = not authenticated (no/invalid credentials — send some). `403` = authenticated but not authorized (credentials fine, permission denied). `502` = the gateway got an invalid response from upstream. `504` = upstream never responded within the timeout.

Q5. HTTP/2 fixed head-of-line blocking. Why did HTTP/3 still need to change transports?

Answer: HTTP/2 fixed HOL blocking at the *application* layer by multiplexing streams, but all streams still ride one TCP connection. TCP guarantees one ordered byte stream, so a single lost packet stalls delivery of every stream behind it. QUIC over UDP implements ordering per-stream, so loss in stream A doesn't block stream B.

Q6. What is forward secrecy and why does TLS 1.3 mandate it?

Answer: Session keys are derived from *ephemeral* key pairs discarded after the session, so compromising the server's long-term private key later cannot decrypt recorded past traffic. TLS 1.2's RSA key exchange lacked this — an attacker could record traffic for years and decrypt it all upon stealing one key. TLS 1.3 removed RSA key exchange entirely.

Q7. Why is consistent hashing preferred over `hash(key) % N` for a cache cluster?

Answer: With modulo, changing N remaps almost every key (4→5 servers remaps ~80%), causing a mass cache miss and a stampede on the database. Consistent hashing places servers on a ring so adding/removing a node only remaps the keys in that node's arc — roughly 1/N of keys.

Q8. A reverse proxy terminates TLS. Your app now logs the proxy's IP for every user and generates `http://` links. Why, and how do you fix it?

Answer: The proxy opens a *new* connection to the backend, so the backend's socket peer is the proxy and the scheme is plain HTTP. Fix: proxy sets `X-Forwarded-For` and `X-Forwarded-Proto`; the app is configured to trust these headers — but only when the request comes from the known proxy IP, or clients could spoof their address.

Q9. When would you deliberately choose UDP over TCP for a service you're building?

Answer: When timeliness beats completeness (voice/video/gaming — a retransmit of stale data is useless), when you need broadcast/multicast, when a single tiny request/response makes a handshake wasteful (DNS), or when you want to implement your own reliability semantics (QUIC, WireGuard).

Q10. Explain why a `Vary: User-Agent` header can wreck a CDN.

Answer: `Vary` adds the listed header to the cache key. There are millions of distinct User-Agent strings, so the CDN stores a near-duplicate copy per browser build. Hit rate collapses and nearly every request goes to origin.

Scenario

Q11. A page takes 4 s to load. `curl -w` shows: DNS 15 ms, connect 30 ms, TLS 60 ms, TTFB 3.6 s, total 3.9 s. Where's the problem?

Answer: Networking is fine — 105 ms to first byte *request*. TTFB of 3.6 s is server-side processing: slow database query (missing index, N+1), a blocking external API call, or no caching. Profile the application; the network layer needs no work.

Q12. You deploy a new version. Users on some servers see the old version, and some report being logged out. What went wrong?

Answer: Two issues. (a) A rolling deploy is in progress and the LB is still routing to un-updated nodes — use connection draining and a proper rollout. (b) Sessions are stored in server memory with sticky sessions; when a node was replaced, those sessions died. Fix: externalize session state to Redis or use signed JWTs so any node can serve any user.

Q13. You changed your site's IP but half your users still hit the old server hours later. Why, and what should you have done?

Answer: DNS caching — resolvers and browsers hold the old A record until its TTL expires. Should have lowered the TTL to ~60 s at least 24–48 h *before* the migration (so old high-TTL entries had time to expire), migrated, kept both servers running through the transition, then raised the TTL again.

Q14. Your API works in Postman but the browser shows "blocked by CORS policy." Explain.

Answer: CORS is enforced by browsers only; Postman and curl ignore it. The browser sent a preflight `OPTIONS` (or checked the response of a simple request) and the server didn't return an `Access-Control-Allow-Origin` matching the page's origin. Fix on the *server*: return the correct `Access-Control-Allow-*` headers and handle the `OPTIONS` method. If credentials are involved, the origin must be explicit — `*` is rejected with `Allow-Credentials: true`.

Q15. A health check hits `/health`, which queries the database. The DB has a 10-second hiccup. What happens?

Answer: Every backend fails its check simultaneously, the LB marks the entire fleet unhealthy, and the site returns 503 even for requests that don't touch the DB — a self-inflicted total outage from a partial degradation. Fix: separate liveness (process alive) from readiness, use shallow checks for LB rotation, add failure thresholds/hysteresis, and never let a shared dependency fail every node at once.

Recall drills (write from memory, no hints)

1. The 7 OSI layers, their PDUs, and one protocol each.
2. Draw the TCP 3-way handshake and 4-way teardown with sequence/ack labels.
3. List 10 DNS record types and what each is for.
4. Write out the HTTP status code families plus 8 specific codes with meanings.
5. Draw the TLS 1.3 handshake and state how many RTTs it costs.
6. List 6 load balancing algorithms and when each is appropriate.
7. Explain push vs pull CDN in three sentences.
8. Fill a table: forward proxy vs reverse proxy, 5 rows.
9. Recite the full browser→server lifecycle in under 60 seconds.
10. For each of §1–§8, name where it appears in the lifecycle of §9.

12. Cheat Sheet

Common Ports

Port	Protocol	Port	Protocol
20/21	FTP (data/control)	443	HTTPS / HTTP-3 (QUIC, UDP)
22	SSH / SFTP	465, 587	SMTPS / SMTP submission
23	Telnet (insecure)	853	DNS over TLS
25	SMTP	993 / 995	IMAPS / POP3S
53	DNS (UDP + TCP)	3306	MySQL
67/68	DHCP	5432	PostgreSQL
80	HTTP	6379	Redis
110 / 143	POP3 / IMAP	27017	MongoDB
123	NTP	8080	HTTP alternate

Ranges: **0-1023** well-known · **1024-49151** registered · **49152-65535** dynamic/ephemeral.

Numbers Worth Memorizing

Thing	Value
Ethernet MTU	1500 bytes
TCP header	20-60 bytes
UDP header	8 bytes
IPv4 header	20-60 bytes
IPv4 / IPv6 address	32 bits / 128 bits
MAC address	48 bits
Port number	16 bits (0-65535)
DNS UDP response limit	512 bytes (4096 with EDNS0)
TCP TIME_WAIT	~2×MSL (~60 s on Linux)
TLS 1.2 / 1.3 handshake	2 RTT / 1 RTT (0-RTT resume)
HTTP/1.1 parallel conns per host	~6

Latency Intuition (order of magnitude)

Operation	Time
L1 cache reference	1 ns
Main memory reference	100 ns
SSD random read	100 μ s
Round trip within same datacenter	0.5 ms
Disk seek (HDD)	10 ms
RTT Dhaka $\leftrightarrow$ Singapore	~40 ms
RTT Dhaka $\leftrightarrow$ Europe	~130 ms
RTT Dhaka $\leftrightarrow$ US East	~230 ms

One-line Summaries

- **OSI** — 7-layer mental model; encapsulation adds a header per layer going down.
- **TCP** — reliable ordered byte stream, handshake + ACKs + congestion control. **UDP** — fire and forget, 8-byte header.
- **DNS** — hierarchical, cached name $\rightarrow$ IP lookup over port 53; root $\rightarrow$ TLD $\rightarrow$ authoritative.
- **HTTP** — stateless request/response; HTTPS = HTTP + TLS on 443.
- **TLS** — asymmetric crypto to authenticate + agree a key, then symmetric for the data.
- **Load balancing** — spread traffic across healthy backends; L4 = fast & dumb, L7 = smart & content-aware.
- **CDN** — cache content near the user to defeat the speed of light and shield the origin.
- **Reverse proxy** — an intermediary in front of *servers* doing TLS, caching, routing, and security.
- **Lifecycle** — parse $\rightarrow$ DNS $\rightarrow$ TCP $\rightarrow$ TLS $\rightarrow$ HTTP $\rightarrow$ CDN/LB/proxy $\rightarrow$ app $\rightarrow$ DB $\rightarrow$ render.

Suggested Study Order

1. **OSI** (vocabulary for everything else)
2. **TCP/UDP** (the transport reality)
3. **DNS** (first thing that happens)
4. **HTTP** (what you actually work with daily)
5. **TLS** (what wraps HTTP)
6. **Reverse proxy** $\rightarrow$ **Load balancing** $\rightarrow$ **CDN** (infrastructure, in increasing scope)
7. **Lifecycle** (assembles all of the above — do this last, it's your self-test)

If you can explain §9 fluently to someone else, you've learned this topic.